Chapter 7

Self Organizing Maps

7.1 Introduction

Have you ever tried of thinking of how an N dimensional space would look like? Of course the N dimensional space is something beyond our imagination. Hence a point in the N dimensional plain is also beyond our imagination. But for a mathematician the representation of N dimensional point is very simple; he can represent it in the form of an 1 X N vector.

It is very easy for us to understand and analyze a point (data) in 2 or 3 dimensional space. For example it possible for us to visualize a dot in a plain paper (it is actually a point in 2 dimensional plain), The position of this dot could easily be represented mathematically by an X coordinate value and a Y coordinate value, also it is not much of an issue to visualize the position of a ball in air (which is actually a point in 3 dimensional spaces). The position of the ball in mathematical representation is only a point in 3 dimensional spaces, which will have an X component, Y component and Z component; this is an example of 3 dimensional data. So mathematicians thought of methods by which we can visualize or analyze N dimensional data.

As a consequence of this, Professor *Teuvo Kohonen* – a Finnish academician and researcher developed a method by which an N dimensional data can be better visualized, by projected on to a low dimensional space(usually one or two dimensions) This process is called by the name Self Organizing Maps. SOM is basically an algorithm which allows us to compresses; at the same time organize the N dimensional input data in a lower dimensional space. This data compression technique is known by the name Vector Quantization [3]. It is also known by the name KNN (Kohonen Neural Network).

As said before this algorithm has the capability to classify data without supervision or training process. Thus SOM can be thought of as a higher version of clustering. *More specifically SOM is an unsupervised Artificial Neural Network algorithm*.

NOTE

The SOM works on the neural network platform, where there are N inputs given to the input layer. There should be an N dimensional weight vector .A weight readjustment takes place in the middle layer/layers and the output layer give out the correct classification output. The only difference is that the weights are adjusted based on certain conditions

One of its major advantage is that it allows the visualization of N dimensional data into visually interpretable data. For example: Now for the sake of understanding, let us consider that we have a 3 dimensional input data which has to be mapped to a lower dimensional space i.e. to a 2 dimensional space. The 3 dimensional input data here can be thought of as different color components as shown in Figure 7.1 [Refer accompanied CD for the color picture]. We call the color data as 3 dimensional vector since each color is made of three components Red, Green and Blue. Now look at figure 7.2 (a) - we have a 40 x 40 matrix cells which are assigned with some random colors (weight vectors). The algorithm projects each input vector (given in fig 7.1) on to each of the 40 x 40 array of random weight value and starts to adjust their weight depending on the input vectors.

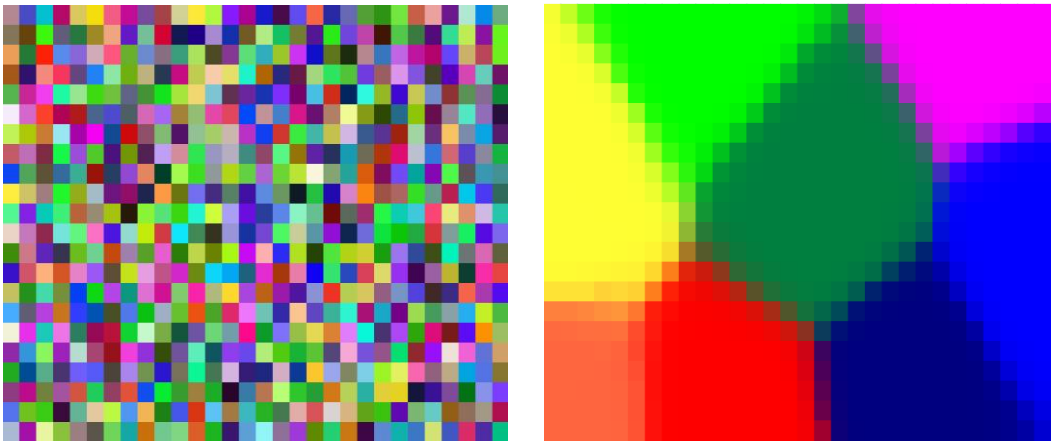
NOTE

The matrix cells actually should have been filled with valid data value (vector) form real datasets. In this example we generated random values as weight vector due to non-availability of valid data.

In this case , the different colors given are red, light green, dark green, light blue, dark blue, yellow, orange and pink as vector inputs. The SOM algorithm will find the best matching color from the random colors (weight vectors) and adjust itself as well as its neighbor colors vector values to make the color more similar to the input color vector. Thus SOM will group the different color to one of the input colors. Ultimately the output will have only 8 different colors. Now by seeing the two dimensional plot one can easily group the weight vectors which are close to the input color vectors. The figure 7.2(b) shows the 8 different colors obtained after self organizing [2]. [To obtain the color images refer attached CD].



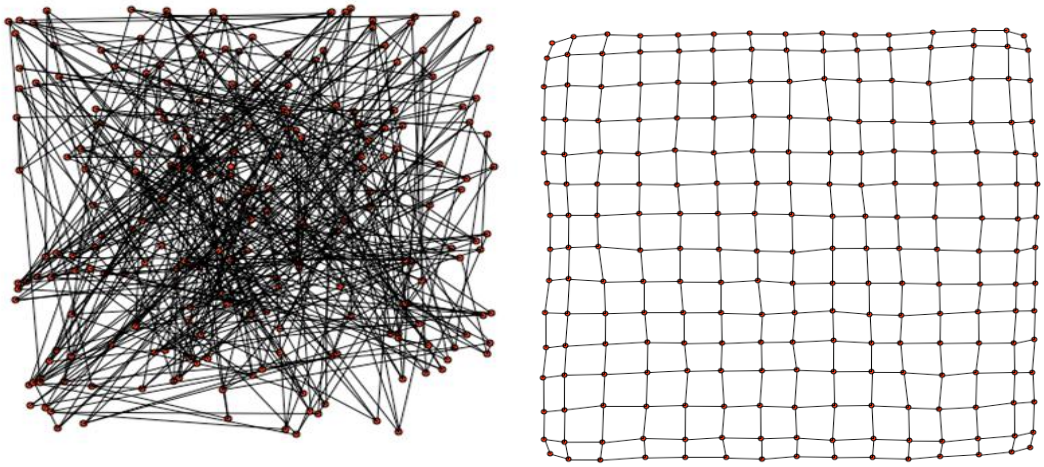
Fig 7.1: Input vectors (3 Dimensional)



a) Random weight vectors b) Self Organized weight vectors

Fig 7.2

The figure 7.3 shows a different scenario where SOM algorithm is used. Here each node of a 15x15 array of nodes, is given a two-dimensional input then plot according to the value of the weights as x-y coordinates. Draw a line from each node to its four neighbors (or three, if it's on the edge. or two, if it's a corner node). With each weight initialized to a random value, the initial graph is a spaghetti mess as in Fig 7.3 (a). Using SOM we will be able to bring together those node which are having connection. This is being displayed in Fig 7.3 (b) [1].



(a) (b)

Fig 7.3: Mapping a square with a two-dimensional lattice

To make things more clear let's take a real dataset and run SOM algorithm on it. The figure 7.4 show the various attributes of districts in Kerala (India).

District	Area	Population	Sex Ratio	Growth rate	Density per Km2
Kasaragod	1992	1203342	1047	12.3	604
Kannur	2966	2412365	1090	7.13	813
Wayanad	2131	786627	1000	17.04	369
Kozhikode	2344	2878498	1058	9.87	1228
Malappuram	3550	3629640	1063	17.22	1022
Palakkad	4480	2617072	1068	9.86	584
Thrissur	3032	2975440	1092	8.7	981
Ernakulam	2407	3098378	1017	9.09	1050
Idukki	5019	1128605	999	6.96	252
Kottayam	2203	1952901	1025	6.76	722
Alappuzha	1414	2105349	1079	5.21	1496
Pathanamthitta	2642	1231577	1094	3.72	574
Kollam	2491	2584118	1070	7.33	1038
Thiruvananthapuram	2192	3234707	1058	9.78	1476

Fig 7.4: various attribute values of all districts in Kerala

Now we want to group districts into three categories:

a) **Category 1**: Area close to 1992, population close to 1100000, Sex ratio close to 1000, Growth rate close to 11, Density close to 500.

b) **Category 2**: Area close to 1400, population close to 2500000, Sex ratio close to 1500, Growth rate close to 6, Density close to 1300.

c) **Category 3**: Area close to 200, population close to 4000000, Sex ratio close to 1100, Growth rate close to 9, Density close to 1500.

So we first create a grid . Fill it with the normalized data provided in figure 7.4 and use SOM algorithm to find these three categories of districts from Kerala map. For that the above said category vectors are passed as inputs to the SOM. The output we obtained is as follows [The implementation is done using macros and can be obtained from the attached CD]. The districts shown in same color are grouped and said to as belonging to one category.

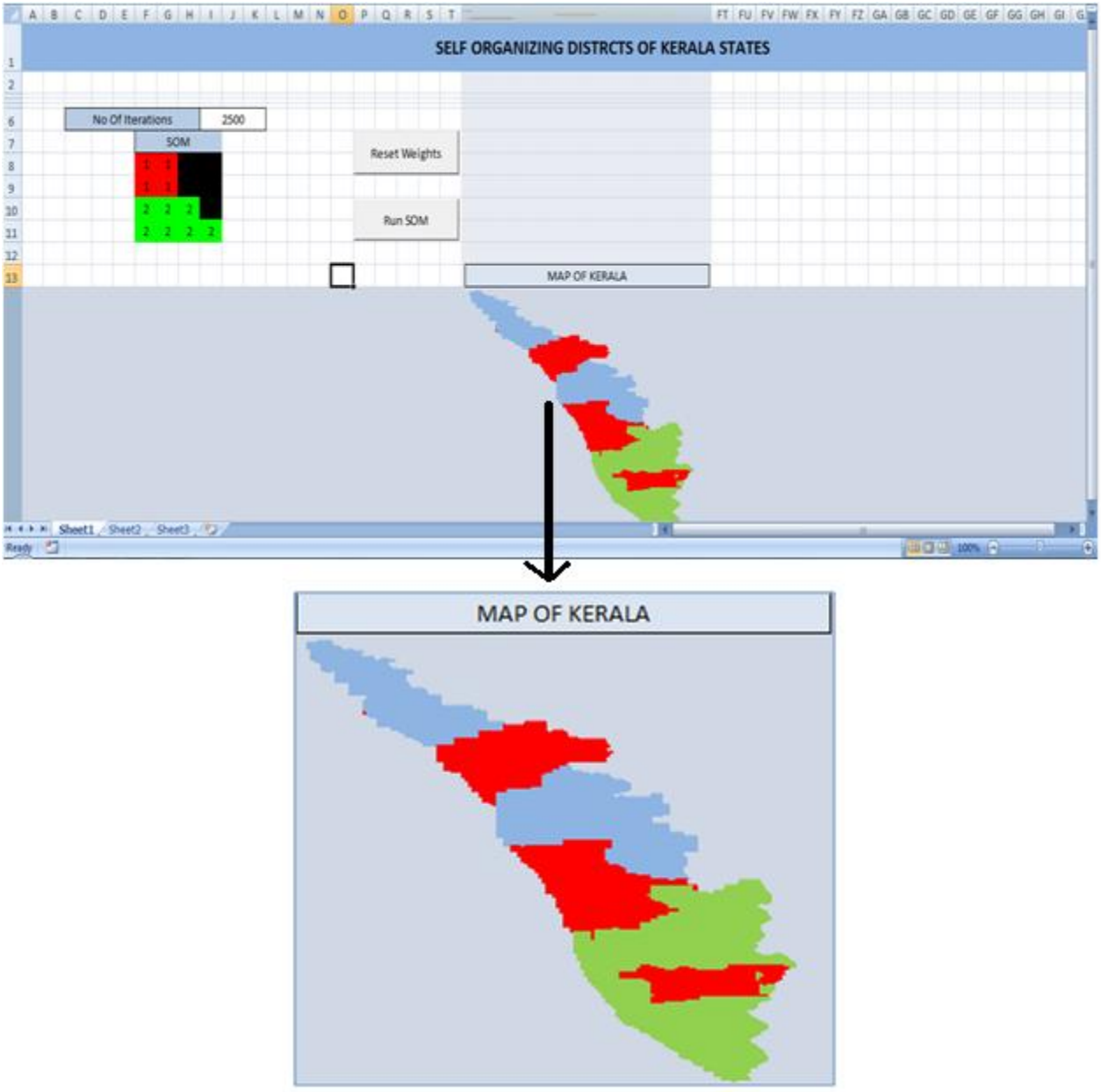


Fig 7.5: Grouped districts (Screenshot from Excel Implementations)

In the top left portion of the figure 7.5, a small square box can be seen. It is made of 16 cells (even though only 15 districts exist in Kerala, to make it a square region we included the extra cell). Each cell is representing a district in Kerala. The cells group in the same color will be those districts which share common features i.e. to same categories.[Obtain color images of above figures from attached CD].

7.2 Abilities of Self Organizing Maps

1) **Approximation of the input space**: The main feature of SOM is its ability to map the data in higher dimension to a two dimensional grid of nodes, where the data can be effectively interpreted or visualized (i.e. it helps in Information Visualization**)**. The collection of nodes (weight vectors) provides a good approximation of the input space. It also preserves the topological relationships of the signal space on to the two-dimensional display.

   An example of dimension reduction technique is latent semantic indexing (Furnas et al., 1988). Latent semantic indexing uses the estimation of latent structure to construct a semantic space that has a much smaller dimensionality than the dimensionality of the original document-term vectors.

2) **Topological ordering**: The SOM algorithm starts with locating the best matching unit among the nodes and then adjusting their weights vector values. Thus spatial location on the lattice corresponds to a certain feature of input patterns. Also near-by neurons on the lattice represent similar input features.
3) **Density matching**: If there is more input vectors which are closer, in the input space, more neurons are used to represent the attribute in mapping plane, to represent their density.
4) **Feature selection**: For displaying the vector effectively, best features to approximate the underlying distribution should be selected.

7.3 Applications of SOM

7.3.1 Data clustering and visualization.

The self organizing maps come to be very useful because N dimensional data/information can be obtained from a 2 dimensional plot of the data. The main application of the SOM is

1) **Detection and Prediction of Salmon Spawning Beds-** Multiple environmental criteria including, river bed material, water depth, and water velocities, affect the location of

salmon spawning beds. Each year Fall Chinook salmon leave the ocean and return to their native freshwater rearing grounds to spawn. Because these species of salmon are listed as a Threatened and Endangered Species in the United States, monitoring the health and populations are required to help understand the stressors. One method of monitoring the populations is to locate and map the spawning beds which are only visible through the water for approximately 3-weeks during spawning period. The current approach for collecting this data is to: 1) conduct aerial surveys of the spawning beds and do visual counts and rough mapping of the areas, and 2) collect aerial imagery and manually digitize the spawning beds. An ongoing study is using the ALCP with a number of supporting datasets, including true color aerial imagery, channel bathymetry, and modeled water velocities, water surface elevations, and water depth, to automatically capture the location and extent of the spawning beds (see Figure 7.6). In addition, the data relationships developed through this process can be used to predict the location of other likely spawning areas.

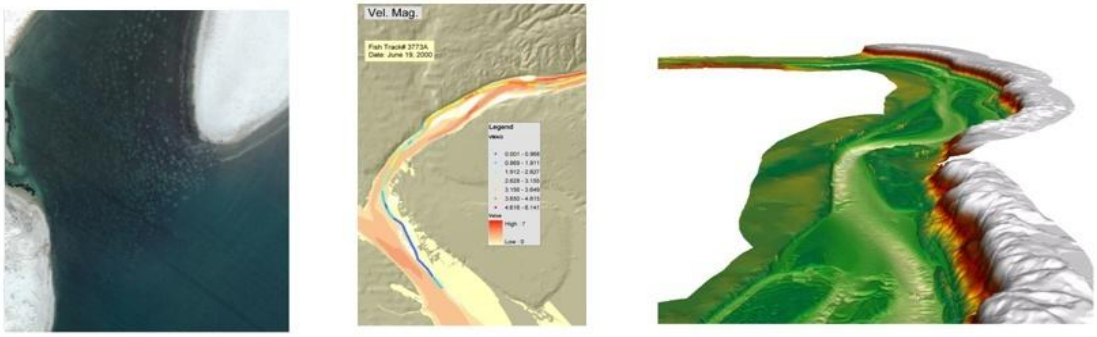


Fig 7.6: Salmon Spawning Beds

2) **Watershed Characterization Mapping - By** understand the hydrologic properties of sub-catchments in an area with limited ground data, it would be possible to predict the hydrological properties of regions having similar properties. The figure 7.7, shows one of that kind.

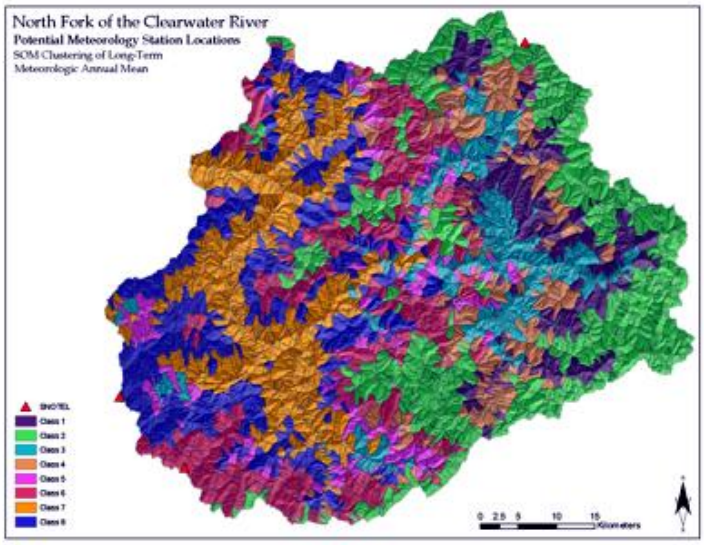


Fig 7.7: Watershed Characterization Map

3) **Determination of Erosion Potential - By using spatially derived data such as** remotely-sensed canopy and ground cover data , various terrain model derived it is possible to predict the probability for soil erosion. Using this data as input, it is possible to predict regions having the same probability of erosion. The figure 7.8 shows the soil erosion probabilities of different regions in north fork of Clearwater river

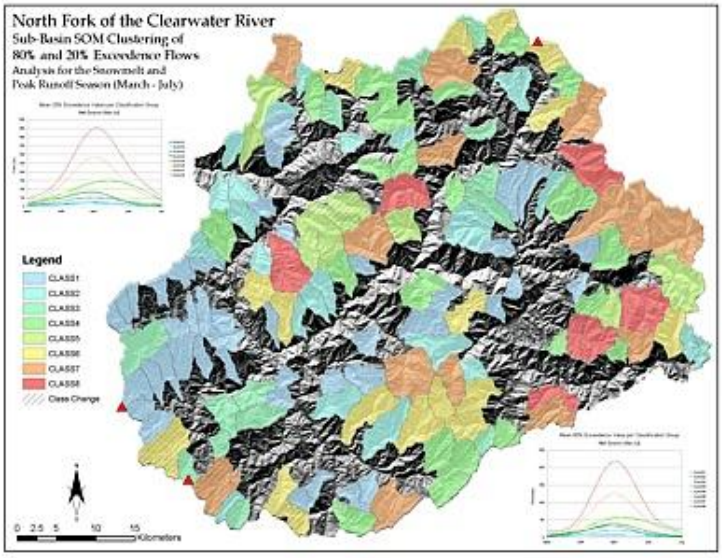


Fig 7.8: Clustered image showing probabilities of soil erosion

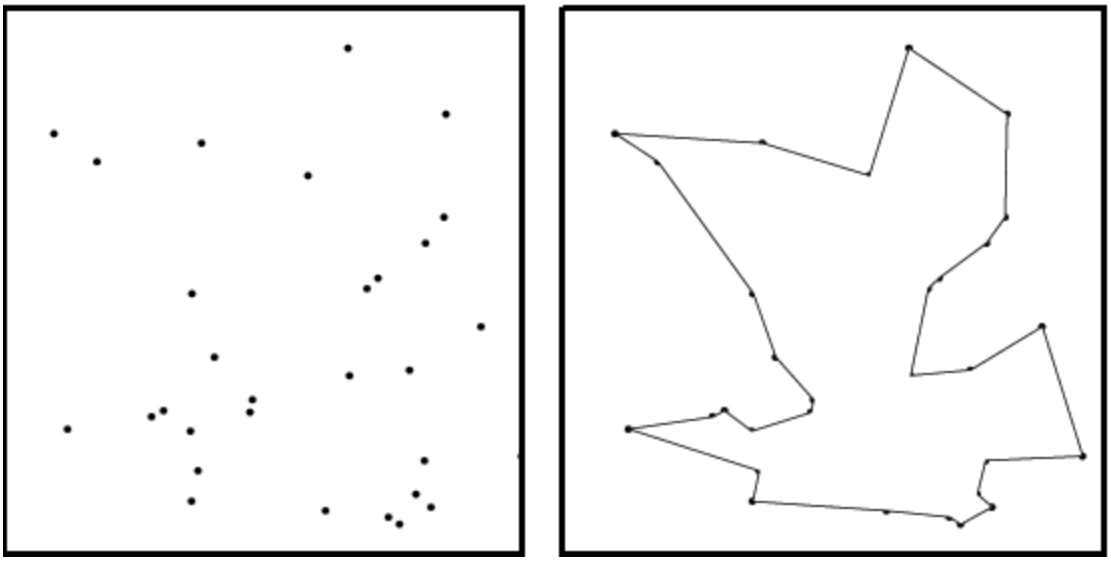
4) Classification of Stocks Using Self Organizing Map- The result shows that the performance of stocks belonging to the best class among the classes generated by SOM gives maximum returns on investment.
5) Determination of the Most Suitable Meteorology Station Data for Bio-fuels Model Input

7.3.2 Optimization problems:

The **Travelling Salesman Problem** is one of the hardest optimization problems which are being studied in operations research and theoretical computer science and it has intrigued mathematicians and computer scientists for years. It is used as a benchmark for many optimization methods. The simplicity of the problem requirement will amaze people for its complexity in obtaining a solution. The travelling salesman problem consists in finding the shortest (or a nearly shortest) path connecting a number of locations (perhaps hundreds), such as cities visited by a travelling salesman on his sales route. This has many applications in science and engineering. For example, in the manufacture of a circuit board, it is important to determine the best order in which a laser will drill thousands of holes. An efficient solution to this problem reduces production costs for the manufacturer. It appears as a sub-problem in many areas, such as DNA sequencing.

The problem was first formulated as a mathematical problem in 1930 and is one of the most intensively studied problems in optimization. It is where the neural networks come with a helping hand. The elastic net is used for solving the traveling salesman problem (TSP) When we represent the location of each city by a point V^{μ} in the two-dimensional input space $\mathrm{V} \in \mathrm{R}2$, and a tour by a sequence of N neurons—which comprise a ring or closed chain A—then a solution to the TSP can be envisaged as a mapping from V-space onto the neurons of the chain. Evidently, we expect the neuron weights to coincide with the input points (‘‘cities’’) at convergence [4].

The points in figure(7.9 (a)) represent cities and are chosen randomly from the input distribution demarcated by the square box and the figure (7.9(b)) shows the optimal route obtained.



(a)Cities **(b) Shortest route**

Fig 7.9

7.3.3 **Semantic maps (**Natural language processing)

The self-organizing map has the property of effectively creating spatially organized internal representations of various features of input signals and their abstractions. One result of this is that the self-organization process can discover semantic relationships in sentences. The semantic relationships in the data are reflected by their relative distances in the kohonen map. It helps one to visually organize and graphically show the relationship between one piece of information and another. This ability of semantic maps can be extended, to group documents based on the document titles.

Preprocessing of text is required before applying it to SOM. Preprocessing mainly refers to removal of non textual content from the documents headings, excluding some stop words, least frequently occurring words, the most frequently occurring words and removal of those words which can rarely be keywords. [Mostly those would the words that are not repeated more than a threshold number of times]. Also sometimes common words that are not supposed to discriminate any discussion topics are also discarded from the documents. Ultimately preprocessing aims at getting N (let N= 25) number of words which is possibly found in the document titles and they are used as the set of indexing words for this collection.

In SOM we require a feature map of appropriate size and input vectors to train the feature map. Assume we have 150 documents to be mapped; the input document vector obtained for each of the document is a vector of size 1 x 25. For this reason a feature map of size not less than 150 x 25 is created. Now the document vectors are used as input to train a feature map of 25 features and 150 nodes. Each node on the map is associated with a vector of weights which are assigned small random values at beginning of the training. During the training process, a document is randomly selected, the node closest to it in N-dimensional vector distance is chosen; the weight of the node and weights of its neighboring nodes are adjusted accordingly. The training process proceeds iteratively for a certain number of training cycles. It stops when the map converges. When the training process is completed, each document is mapped to a grid node.

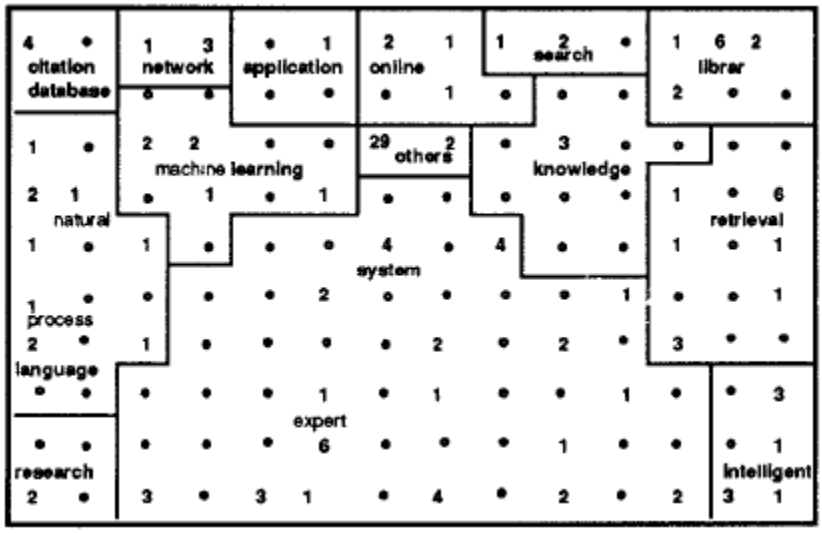


Fig 7.10: The areas on the map, their relative positions, neighbors, and sizes are determined by the input document vectors. The number on the map represents the number of documents mapped to each node.

Fig 7.10 is the semantic map of documents obtained after 2500 training cycles. The map contains very rich information. The numbers on the map indicate the number of documents mapped to their corresponding nodes. These numbers collectively reveal the distribution of the documents on the map. Also the size of the areas corresponds to the frequencies of occurrence of the words [5][6].

The use of self-organized maps in practical speech recognition and a simulation experiment on semantic mapping are discussed

7.4 The Algorithm

The SOM is the most powerful among the unsupervised classification algorithms.

1. First of all, each node's weights are initialized with a random value.
2. A vector is chosen at random from the set of training data and presented to the lattice.
3. Every node is examined to calculate which one's weights are most like the input vector. The winning node is commonly known as the Best Matching Unit (BMU).
4. The radius of the neighborhood of the BMU is now calculated. This is a value that starts large, typically set to the 'radius' of the lattice, but diminishes each time-step. Any nodes found within this radius are deemed to be inside the BMU's neighborhood.
5. Each neighboring node's (the nodes found in step 4) weights are adjusted to make them more like the input vector. The closer a node is to the BMU; the more its weights get altered.
6. Repeat step 2 for N iterations [2] .

NOTE
The SOM is one of the most realistic models of the brain function.

7.5 Dimensionality Reduction using SOM (using macros)

The SOM is basically a dimensionality reduction method which aims at representing a higher dimensional data on a lower dimensional space. Here we have a 200 x 200 matrix (which in our case is a range of cell) with randomly assigned weight values (RGB vector). We have 8 different colors as the input vectors. Now the algorithm tries to adjust the randomly assigned weight vectors (color) to the above assumed eight different colors.

STEP1: Input Vector Selection & Weight Initialization

Here the different input color vectors given are red, light green, dark green, light blue, dark blue, yellow, orange and pink. Just for visualization pupose one can use *Range().Interior.Color* function available in MS Excel to print the color of interest on the top of excel sheet 1. This is done just for making the user informed about the different color vectors which are provided as input to SOM.

Now for the whole 200 x 200 cell range, we color the cells randomly using the below code. Here the "Rnd()" function available with Excel generates random values between 0 and 1. This is multiplied with 255 to get a value between 0 and 255 . We have colored the cell by such randomly generated values for red, green and blue which are given as argument to the RGB function. The 'Int" function is used to round off the randomly generated pixel value to the nearest integer value. The pixel values for red, green and blue, for each cell is stored in an array named *SOM* for future reference of the values [2].

```
For i = 1 To n
    For j = 1 To m
       R = Rnd()
       G = Rnd()
       B = Rnd()
       SOM(i, j, 1) = R
       SOM(i, j, 2) = G
       SOM(i, j, 3) = B
       Range("Som").Cells(i, j).Interior.Color = RGB(Int(R * 255), Int(G * 255), Int(B * 255))
    Next j
Next i
```

STEP2: Locating the best matching unit

The best matching unit is the node for which the Euclidian distance between its weight vector and the current input vector is a minimum. The equation for Euclidian distance calculation as shown in equation 7.1,

$$Dist = \sqrt{\sum_{i=0}^{n} \left(V_i - W_i\right)^2} \qquad (7.1)$$

Where V is the input vector and W is the weight. The corresponding code for finding node with which , the minimum Euclidian distance is found is shown below.

```
For i = 1 To n
        For j = 1 To m
           dist = 0
           For k = 1 To dims
              dist = dist + (SOMSet(whichVec, k) - SOM(i, j, k)) ^ 2
           Next k
           If dist < Min Then
              Min = dist
              Row = i
              Column = j
           End If
        Next j
 Next i
```

SOMSet is the array which stores the R G and B values of input vectors which is initialized in the first step. Here the 'Row' and 'Column' value stores the location of the best matching unit in the array.

NOTE

The dimension of weight vector should be the same as that of the dimension of input vector.

STEP 2: Determining the Best Matching Unit's Local Neighborhood

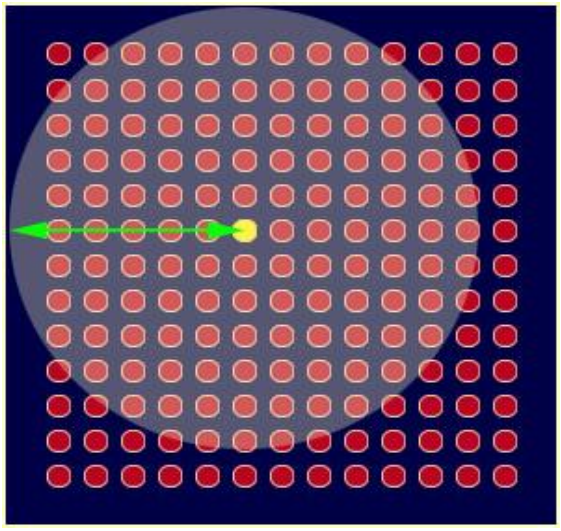


Fig 7.11: The shaded circular region forms the initial neighborhood of the BMU

The next step is to find out the set of nodes which lie within the Best Matching Unit neighborhood and for this the neighborhood has to be defined. The initial radius of the circle around the best matching unit is found out as follows.

If somR.Height > somR.Width Then
mapRadius = somR.Height / 2
Else
mapRadius = somR.Width / 2
End If

Here 'somR' is the range of cells in the excel (in our case it is 200 x 200 matrix range). The *Height() and* Width() gives the length and breadth of the selected range (I.e *somR*) .Once we set an initial radius the set of nodes inside the circle which belong to the cluster can be found out by applying Pythagoras theorem in each of the nodes. A unique feature of the Kohonen learning algorithm is that the area of the neighborhood shrinks over time. This is accomplished by making the radius of the neighborhood shrink over time (with number of iterations) by the exponential decay function given by equation 12.2. [2]

$$\sigma(t) = \sigma_0 e^{\left(-\frac{t}{\lambda}\right)} \tag{7.2}$$

Where t =1, 2, 3 etc. "σ_0"denotes the width of the lattice at time t_0 and the Greek letter lambda denotes a time constant, "t" is the current time-step (iteration of the loop). The corresponding code here for finding neighborhood radius (curRadius) is

$$curRadius = mapRadius * \exp\left(\frac{-iter}{timeConst}\right) \tag{7.3}$$

Where, t is the **timeConst = numIter / (Log(mapRadius) / Log(10)),** "iter" refers to the number of iterations and the "mapRadius" is obtained in the previous step.

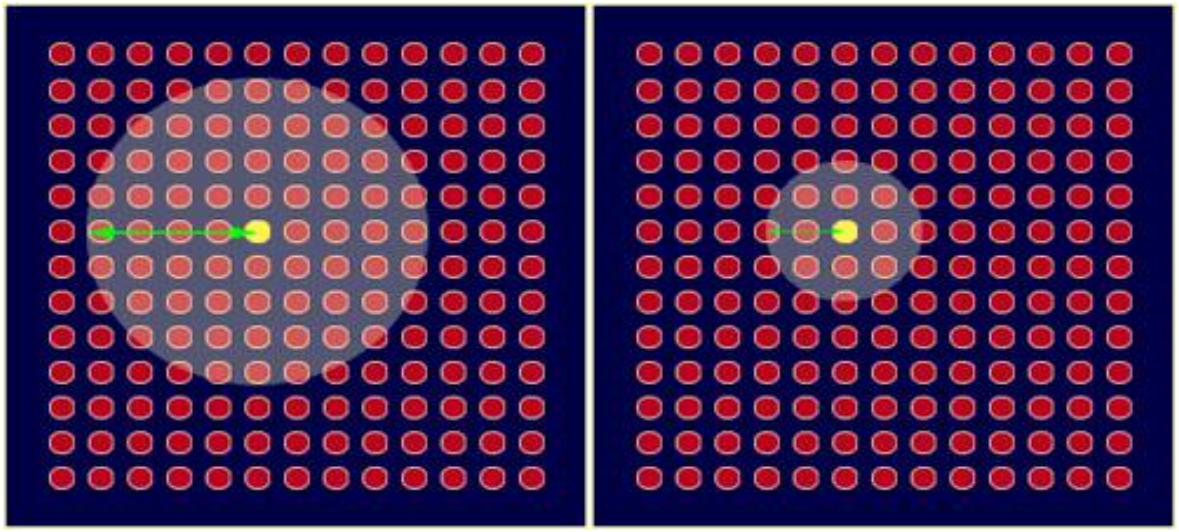


Fig 7.12: Shrinking radius

STEP 3: Find the Learning Rate

The decay of the learning rate is calculated in individual iteration using the following equation. Here as the number of iteration increases i.e. "numIter", the value of "curLearn" decreases. Hence learning rate decreases with each iteration [2].

$$curLearn = learn * \exp\left(\frac{-iter}{numIter}\right) \tag{7.4}$$

The influence a node's distance from the BMU has on its learning. Θ (*t*) is given by equations 7.5

$$\Theta(t) = e^{\left(-\frac{dist^2}{2\sigma^2(t)}\right)} \quad \text{Where t =1, 2, 3...} \tag{7.5}$$

STEP 4 : Adjust the weights

The difference between "SOMSet" and "SOM" for a particular value of "whichVec" which is obtained by equation 7.7,

$$\textbf{whichVec = Int(Rnd() * clust + 1),} \tag{7.7}$$

The 'whichVec' has to give a value between 1 and 8. The above equation is just an implementation of the logic to randomly generate the values between 1 and 8. [Here 'clust' is a constant, which we have assumed as 8]. Now the new weights are found out by adding the

original weight values SOM with a fraction of the output obtained from the above difference. The fraction to be taken is determined by the “theta” and the “curlearn” factors.The below code implements the above logic [2]. Theta is obtained from equation 7.6

$$theta = exp\left(\frac{-distXY^2}{2 * curRadius^2} \right) \tag{7.6}$$

```
For i = 1 To n
For j = 1 To m
distXY = Sqr((x - coords(i, j, 1)) ^ 2 + (y - coords(i, j, 2)) ^ 2)
if distXY <= curRadius Then
theta = Exp(-distXY ^ 2 / (2 * curRadius ^ 2))
For k = 1 To dims
SOM(i, j, k) = SOM(i, j, k) + theta * curlearn * (SOMSet(whichVec, k) - SOM(i, j, k))
Next k
End If
Next j
Next i
```

Basically in the above said step, the adjustment of pixel values for those pixels which are present within the, best matching unit’s local neighborhood is found out.

STEP 6: Coloring Cells

We finally obtain the new SOM matrix values, which are the new color pixel values are used to recolor the matrix range [2].

```
For i = 1 To n
For j = 1 To m
somR.Cells(i, j).Interior.Color = RGB(Int(SOM(i, j, 1) * 255), Int(SOM(i, j, 2) * 255), Int(SOM(i, j, 3) * 255))
Next j
 Next i
```

As said ,In this case all the above steps are done in excel using Excel macro. The figure 7.13 shows the final map obtained by readjusting the random pixel values (selected initially in the 200 x 200 cell region) to different input color pixel values. (Obtain the implementation file (Excel) from the CD.)

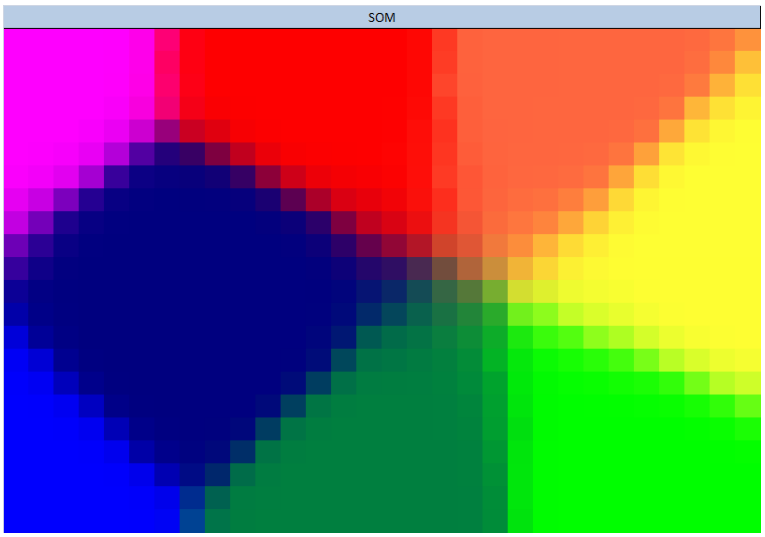


Fig 7.13: Self organized map

7.6 Summary

SOM is the most popular unsupervised learning algorithm among the unsupervised learning algorithms. The ability of algorithm to map a higher dimensional data to a low dimensional space is its power. Now it is used in signal processing, control theory, images and acoustic patterns, adaptive robotic controls, error tolerant transmission of signals, demodulation of signals in telecommunication etc.

Questions:

1) What is SOM? What are its features? What are its pros and cons?
2) Draw the flow of self organizing algorithm? Give the physical interpretation in each step of the algorithm?
3) Give some areas of application for SOM?
4) What is travelling sales man problem? How can it be solved?
5) How is SOM advantageous in remote sensing applications?

References

[1] R. Rojas: Neural Networks, Springer-Verlag, Berlin, 1996

[2] Kohonen's Self Organizing Maps: http://www.ai-junkie.com/ann/som/som1.html

[3] T.Kohonen, "Self Organizing maps", 3rd edition

[4] Marc M. Van Hulle. Kernel-Based Topographic Maps: Theory and Applications. In Collection of Wiley Encyclopedia of Computer Science and Engineering, 2008.

[5] Xia Lin, “A self Organizing Semantic Map for Information Retrieval” College of Library and Information Services, University of Maryland, ISBN:0-89791-448-1.

[6] K. Lagus, T. Honkela, S. Kaski and T. Kohonen, Selforganizing maps of document collections: a new approach to interactive exploration. In: E. Simoundis, J. Han and U. Fayyad (eds), Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (AAAI Press, Menlo Park, 1996) 238–43.